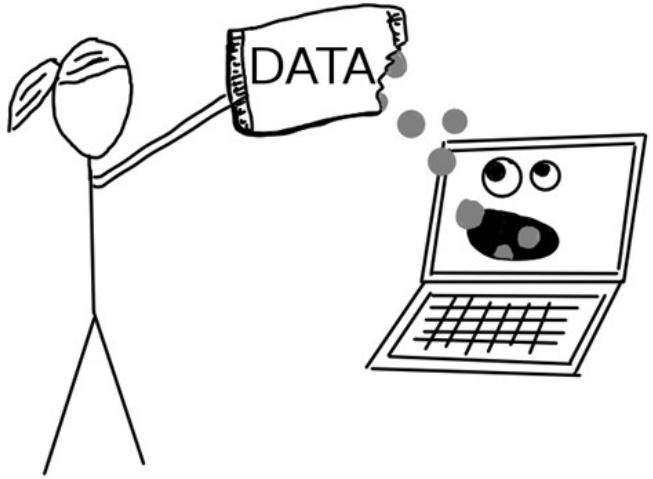
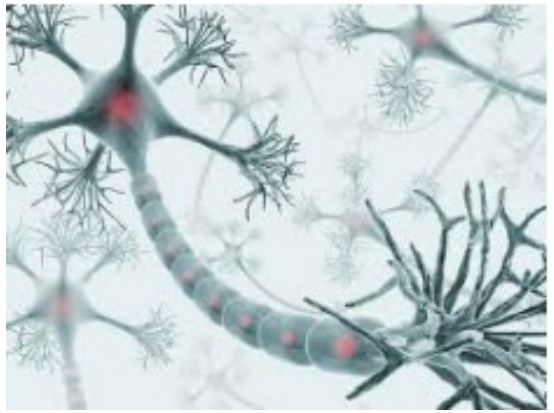




Neural Network

BackPropagation Algorithm

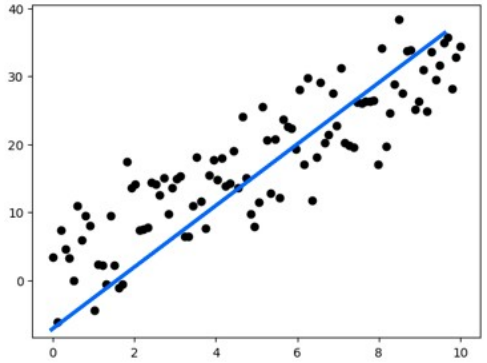
Version 2023

Abdelwahab Naji - Enseignant chercheur
Abdelwahab.naji@gmail.com
ENSET de Mohammedia, Université Hassan II de Casablanca
https://www.youtube.com/c/AbdelwahabNaji

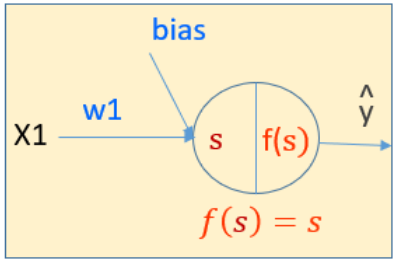
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

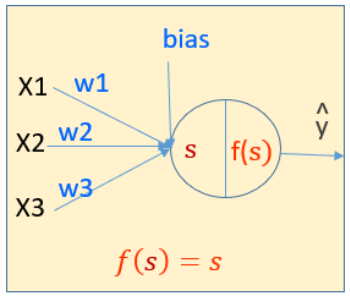


2

Multi Variate Linear Regression Problem

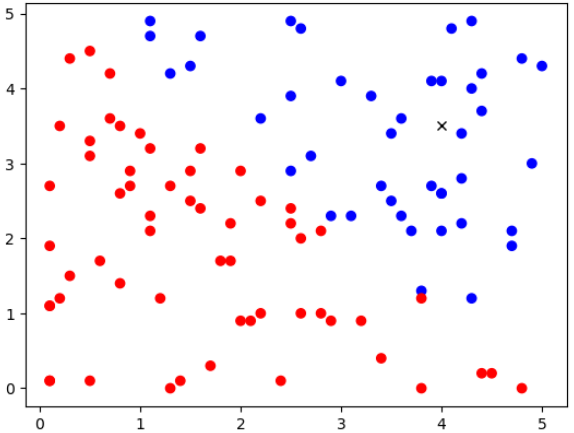
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising problem

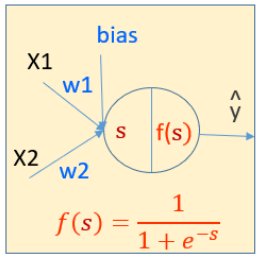


3

Linearly Separable problem

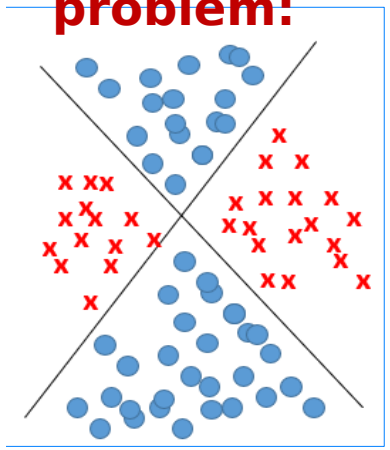


Red or Blue?

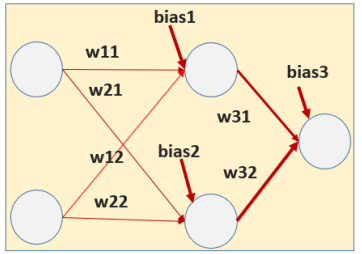


4

Non Linearly Separable problem:

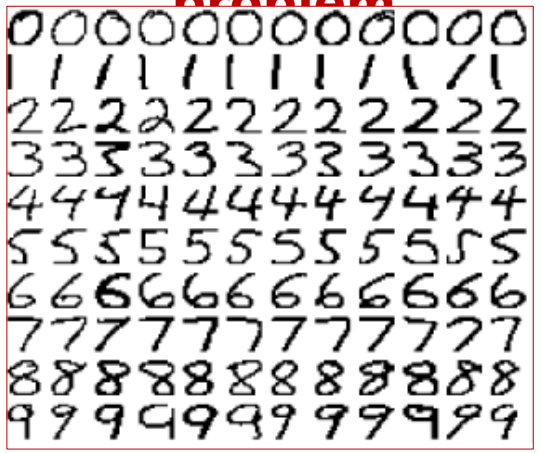


XOR Problem

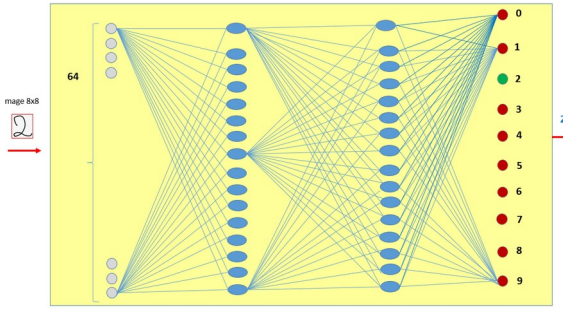


5

Complex classification problem



Hand Written digits

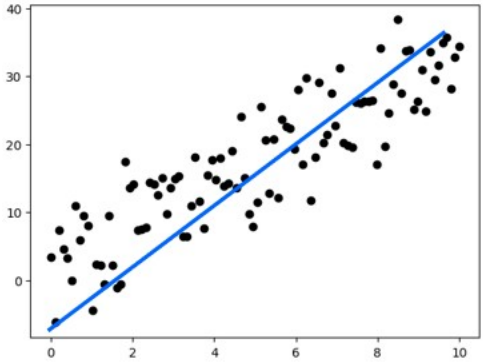


Rappel 1

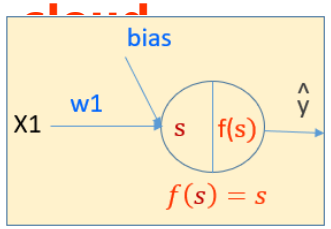
Un réseau de neurones avec un seul neurone pour la régression

1

Simple Linear Regression Problem



Random point cloud




```
def linear(z):
    return z
class NN1Neurone:
    def __init__(self):
        self.lr=0.001
        self.epochs=10000

    def fit(self,X,y):
        self.bias=0.7
        self.w1=0.1
        n=len(y)
        for i in range(self.epochs):
            ws=self.bias+self.w1*X
            py=linear(ws)
            dbias=-(2/n)*sum(y-py)
            dw1=-(2/n)*sum(X*(y-py))
            self.w1=self.w1-dw1*self.lr
            self.bias=self.bias-dbias*self.lr
    def predict(self,X):
        ws=self.bias+self.w1*X
        return linear(ws)
    def getDetail(self):
        return (self.bias,[self.w1])
```

Rappel 2

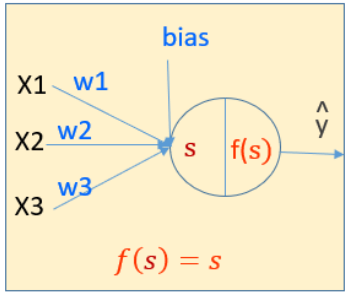
Un réseau de neur
la clas ication

2

Multi Variate Linear Regression Problem

TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
…			
199.8	2.6	21.2	**10,6**

Advertising p




```
class NN1Neurone:
    def __init__(self):
        self.lr=0.001
        self.epochs=10000

    def fit(self,X,y):
        self.bias=np.random.uniform(-1,1)

self.W=np.array(np.uniform(-1,1,X.sha
pe[1]))
        n=len(y)
        for i in range(self.epochs):
            ws=self.bias+np.dot(X,self.W)
            py=linear(ws)
            dbias=-(2/n)*sum(y-py)
            dW=-(2/n)*np.dot(X.T, (y-py))
            self.W=self.W-dW*self.lr
            self.bias=self.bias-dbias*self.lr
    def predict(self,X):
        ws=self.bias+np.dot(X,W)
        return linear(ws)
    def getDetail(self):
        return (self.bias,[self.W])
```

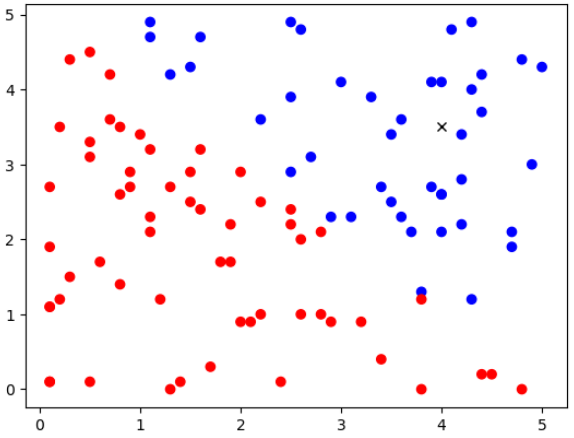
```
class NN1Neurone:
    def __init__(self):
        self.lr=0.001
        self.epochs=10000

    def fit(self,X1,X2,X3,y):
        self.bias=0.7
        self.w1=0.1
        self.w2=0.1
        self.w3=0.1
        n=len(y)
        for i in range(self.epochs):
            ws=self.bias+self.w1*X1
+self.w2*X2 +self.w3*X3
            py=linear(ws)
            dbias=-(2/n)*sum(y-py)
            dw1=-(2/n)*sum(X1*(y-py))
            dw2=-(2/n)*sum(X2*(y-py))
            dw3=-(2/n)*sum(X3*(y-py))
            self.w1=self.w1-dw1*self.lr
            self.w1=self.w1-dw1*self.lr
            self.w1=self.w1-dw1*self.lr
            self.bias=self.bias-dbias*self.lr
    def predict(self,X):
        ws=self.bias+self.w1*X1 +self.w2*X2
+self.w3*X3
        return linear(ws)
    def getDetail(self):
```

Rappel 3

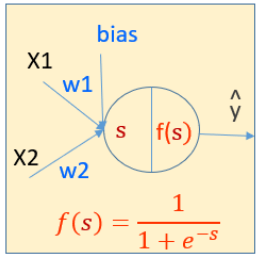
Un réseau de neurones avec un seul neurone pour la classification

3

Linearly Separable problem



Red or Blue?




```
def sigmoid(z):
    return 1/(1+np.exp(-z))
class NN1Neurone:
    def __init__(self):
        self.lr=0.001
        self.epochs=10000
    def fit(self,X1,X2,y):
        self.bias=0.7
        self.w1=0.1
        self.w2=0.1
        n=len(y)
        for i in range(self.epochs):
            ws=self.bias+self.w1*X1+self.w2*X2
            py=sigmoid(ws)
            dbias=-(2/n)*sum(y-py)
            dw1=-(2/n)*sum(X1*(y-py))
            dw2=-(2/n)*sum(X2*(y-py))
            self.w1=self.w1-dw1*self.lr
            self.w2=self.w2-dw2*self.lr
            self.bias=self.bias-dbias*self.lr
    def predict(self,X1,X2):
        ws=self.bias+self.w1*X1+self.w1*X2
        proba=sigmoid(ws)
        cl=[1 if i>0.5 else 0 for i in proba]
        return cl
    def getDetail(self):
        return (self.bias,[self.w1,self.w2])
```

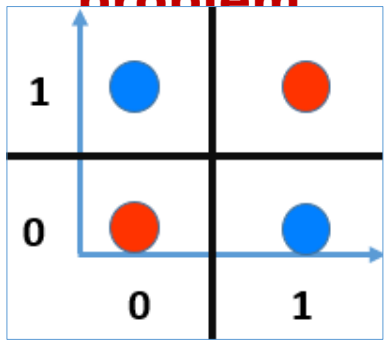
Rappel 4

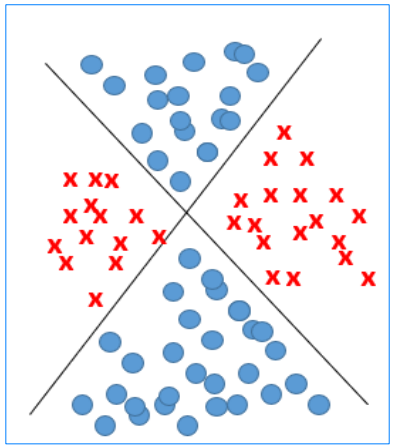
Un réseau de neurones pour la classification d'un problème non linéairement séparable

XOR Problem

4

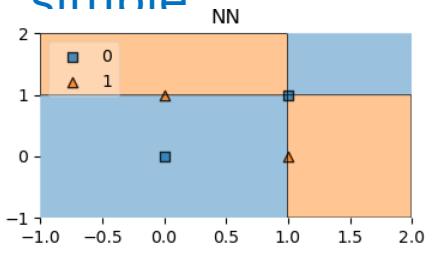
Non Linearly Separable problem:

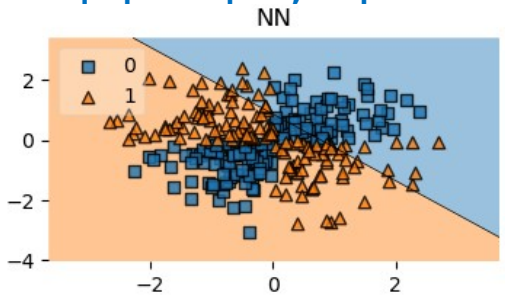


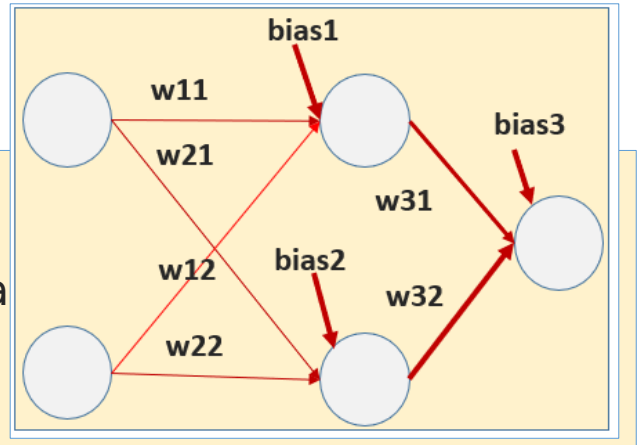


Accuracy=1, vu que le dataset est simple

Accuracy=0.583, vu que le








```
#1---------------Librairies
import numpy as np
from keras.models import Sequentia
from keras.layers import Activation,
import tensorflow as tf

#2----------------Model creation: Neural network architecture
model = Sequential()
hidden_layer = Dense(2,input_dim=2,
kernel_initializer='ones',bias_initializer='zeros',
activation='relu')
output_layer =
Dense(1,kernel_initializer='random_u
bias_initializer='zeros', activation='sigmoid')
model.add(hidden_layer)
model.add(output_layer)
#3-----------------Model compilation: loss, optimizer
#opt=tf.keras.optimizers.SGD(learning_rate=0.001)
#opt=tf.keras.optimizers.Adam(learning_rate=0.001)
opt=tf.keras.optimizers.SGD(learning_rate=0.01,momentum=0.9)
model.compile(loss='binary_crossentropy',optimizer=opt,metrics=['accuracy'])
#4                                   Model training: weights fitting
```

L'accuracy peut être influencé par les poids initiaux

Rappel 5

Comment optimiser la performance d'un réseau neurones

Plusieurs questions sont à soulever:

- Quelle l'architecture du réseau de neurones à adopter
- Comment initialiser les poids
- Quelle fonction loss à choisir
- **Quel optimizer choisir**
- Comment définir les paramètres de l'optimizer choisi
- Comment bien choisir les fonctions d'activation

```
#1---------------Librairies
import numpy as np
from keras.models import Sequential
from keras.layers import Activation, Dense
import tensorflow as tf

#2----------------Model creation: Neural network architecture
model = Sequential()
hidden_layer = Dense(2,input_dim=2,
kernel_initializer='ones',bias_initializer='zeros', activation='relu')
output_layer = Dense(1,kernel_initializer='random_uniform',
bias_initializer='zeros', activation='sigmoid')
model.add(hidden_layer)
model.add(output_layer)
#3-----------------Model compilation: loss, optimizer
#opt=tf.keras.optimizers.SGD(learning_rate=0.001)
#opt=tf.keras.optimizers.Adam(learning_rate=0.001)
opt=tf.keras.optimizers.SGD(learning_rate=0.01,momentum=0.9)
model.compile(loss='binary_crossentropy',optimizer=opt,metrics=['
accuracy'])
#4-------------------------------  Model training: weights fitting
#model.fit(X,y,epochs=1000,batch_size=4)
model.fit(X,y,epochs=1000)
```

Rappel 5

Comment optimiser la performance d'un réseau de neurones

Les différents optimizers:

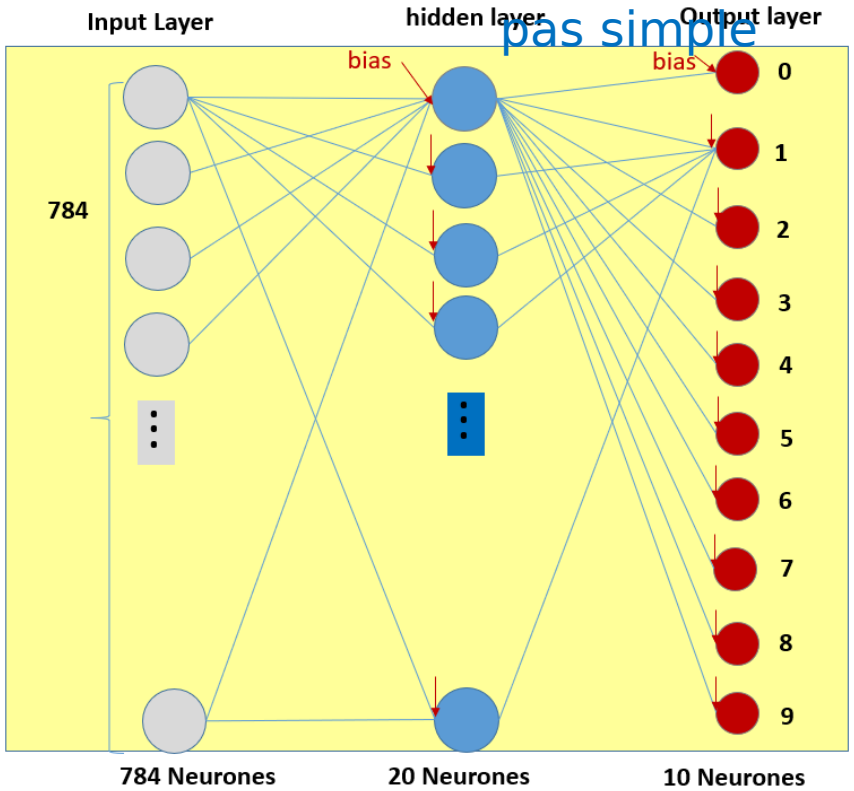
Variant	Advantages	Disadvantages
Batch gradient descent	Guaranteed convergence to global optimum	Computationally expensive for large datasets, slow convergence
Stochastic gradient descent	Faster convergence, more efficient for large datasets	High variance, may not converge to global optimum
Mini-batch gradient descent	Balanced convergence speed and computational cost, efficient for large datasets	Choice of mini-batch size can be a challenge
Momentum gradient descent	Faster convergence, less likely to get stuck in local minima	May overshoot and oscillate around the optimum
Adagrad	Adaptive learning rate, efficient for sparse data	Can stop learning too early
RMSProp	Adaptive learning rate, efficient for non-stationary problems	Can stop learning too early, requires tuning of hyperparameters
Adam	Adaptive learning rate, efficient for large datasets and noisy data	Can converge to suboptimal solutions, requires tuning of hyperparameters

Rappel 6

Créer un réseau de neurones pour la classif

fashion problem

Accuracy=0.8355,
vu que le dataset n'est pas simple




```
import keras
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
dataset = tf.keras.datasets.fashion_mnist
(X_train, y_train), (X_test, y_test) = dataset.load_data()
```

```
model=Sequential()
#couche N°1 : couche d'entree: 28x28, Flatten()
input_layer=Flatten(input_shape=[28, 28])
#couche N°2, couche cachée (20 neurone)
hidden_layer=Dense(20,activation='relu',kernel_initializer='random_uniform')
#couche N°3:  output (10 neurone)
output_layer=Dense(10,activation='softmax',kernel_initializer='random_uniform')
model.add(input_layer)
model.add(hidden_layer)
```

```
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="adam", metrics=["accuracy"])
```

```
model.fit(X_train, y_train,epochs=10)
```

Objectifs à atteindre dans la suite

En tant que Ing Recherche et Dev, comment optimiser son

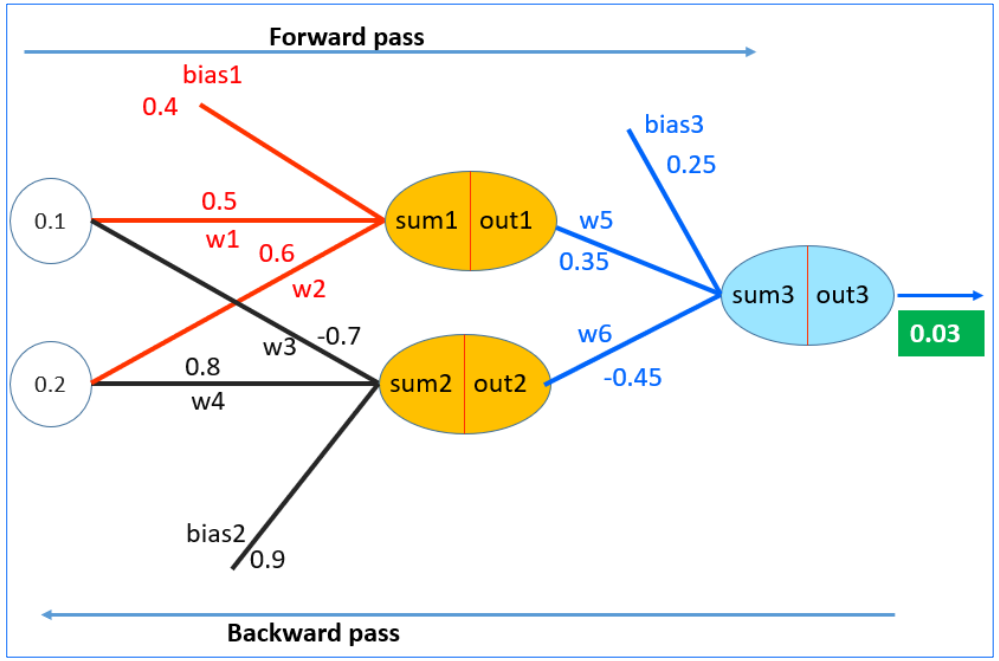
Plusieurs questions sont à soulever:

- Quelle l'architecture du réseau de neurones à adopter
- **Comment initialiser les poids**
- Quelle fonction loss à choisir
- **Quel optimizer choisir**
- Comment définir les paramètres de l'optimizer choisi
- Comment bien choisir les fonctions d'activation
- Quels sont les problèmes récurrents liés aux réseaux de neurones profonds: overfitting, undefitting, gradient explosion, vanishing problem, dataset non équilibré…

Développer un réseau de neurones profond from scratch

Découvrir ce qui est caché!!!

Code source final à obtenir




```
#inputs

X = np.array([[0.1,0.2]])

y = np.array([[0.03]])

learning_rate= 0.1

#weights and bias initialization

hidden_weights = [[0.5,0.6],[-0.7,0.8]]

hidden_bias =[0.4,0.9]

output_weights = np.array([[0.35],[-0.45]])

output_bias = np.array([[0.25]])
```

Forward pass

```
hidden_layer_sum = np.dot(X,hidden_weights) + hidden_bias

hidden_layer_out = sigmoid(hidden_layer_sum)

output_layer_sum = np.dot(hidden_layer_out,output_weights) +
output_bias

predicted_y = sigmoid(output_layer_sum)
```

Backward pass

```
dloss_dw_h= X.T.dot((predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_deriva
tive(hidden_layer_out))

dloss_dbias_h=(predicted_y-y)*
sigmoid_derivative(predicted_y).dot(output_weights.T)*sigmoid_deriva
tive(hidden_layer_out)

dloss_dw_out=hidden_layer_out.T.dot((predicted_y-
y)*sigmoid_derivative(predicted_y))
```

```
#update

hidden_weights-=dloss_dw_h*learning_rate

hidden_bias-=dloss_dbias_h*learning_rate

hidden_bias=np.sum(hidden_bias,axis=0,keepdims=True)

output_weights-=dloss_dw_out*learning_rate

output_bias-=np.sum(dloss_dbias_out,axis=0,keepdims=True)*learning_rate
```

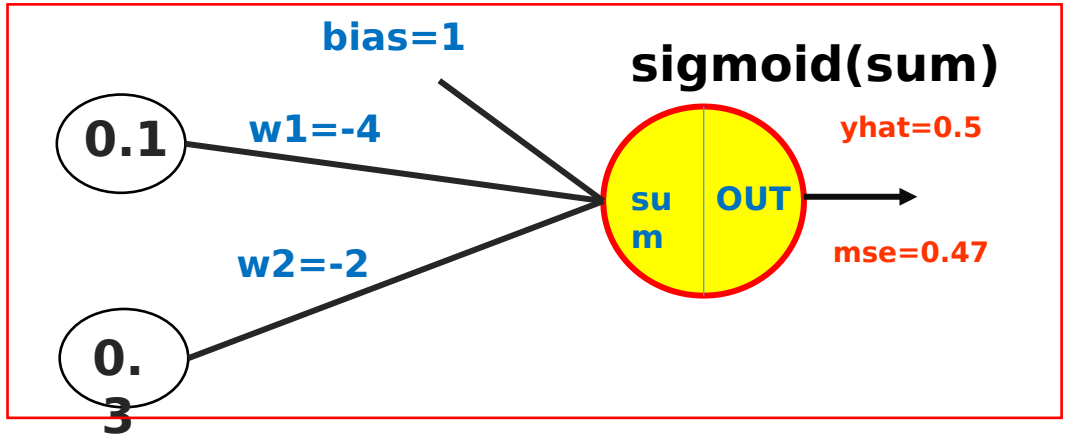
Mise en œuvre d'un Feed Forward Network from scratch:

- Utiliser un dataset simple
- Utiliser une architecture simple avec une seule couche cachée
- Comment ajuster les paramètres d'un réseau de neurones: Appliquer la technique backpropagation
  - Forward pass: prediction
  - Backward pass: ajustement

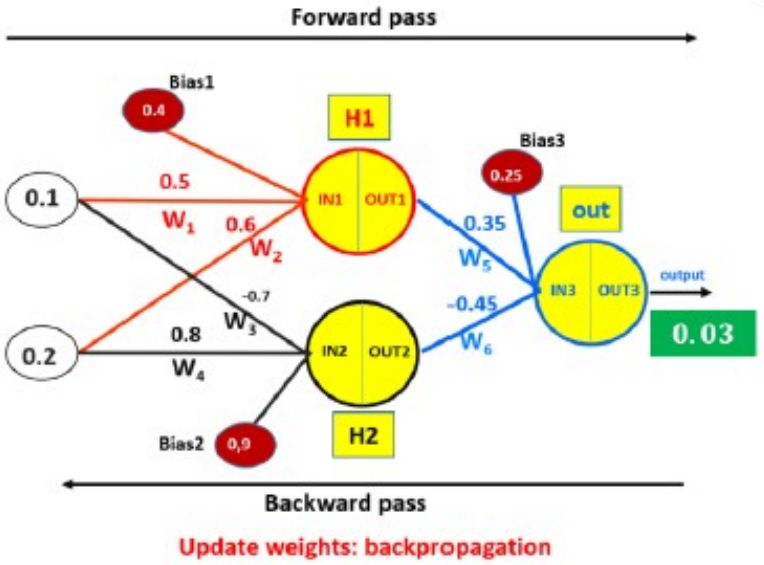
X1	X2	label
0.1	0.3	0.03

Dataset

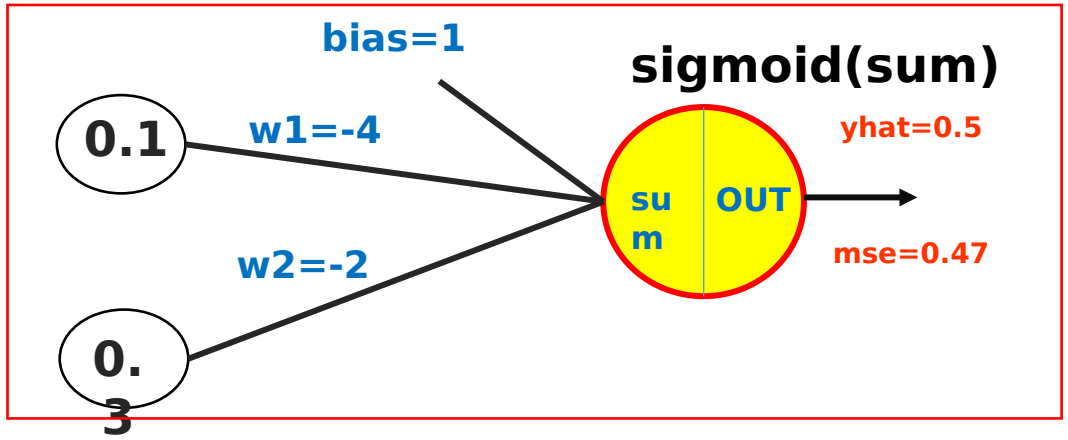
Exemple 1



Exemple 2



Exemple 1




```
def fit(X,y):
    lr=0.01
    epochs=10000
    bias=1
    w1=-4
    w2=-2
    n=1#len(dataset)
    X1,X2=X[0],X[1]
    for i in range(epochs):
        #forward pass
        ws=bias+w1*X1+w2*X2
        yhat=sigmoid(ws)
        #backward pass
        print("iteration: ",i,"===> error=",(y-yhat))
        dbias=-(2/n)*(y-yhat)
        dw1=-(2/n)*(X1*(y-yhat))
        dw2=-(2/n)*(X2*(y-yhat))
        w1=w1-dw1*lr
        w2=w2-dw2*lr
        bias=bias-dbias*lr

    return bias,w1,w2
```

Forward pass, backward pass sans backpropagation technique

1 Initialize weights

Pour i=1 à nbEpochs fair

2

$$sum = \sum_{i=1}^{n} W_i * X_i + bias$$

3 Activation function →

$$f(s) = \frac{1}{1+e^{-s}} = output$$

4 error= desired-predicted

$$Loss = \frac{1}{n}(desired - predicted)^2$$

Avec n=1
Une seule ligne

5 Weights update

$W_i = w_i +$ (desired-predicted)* Xi * learning_rate

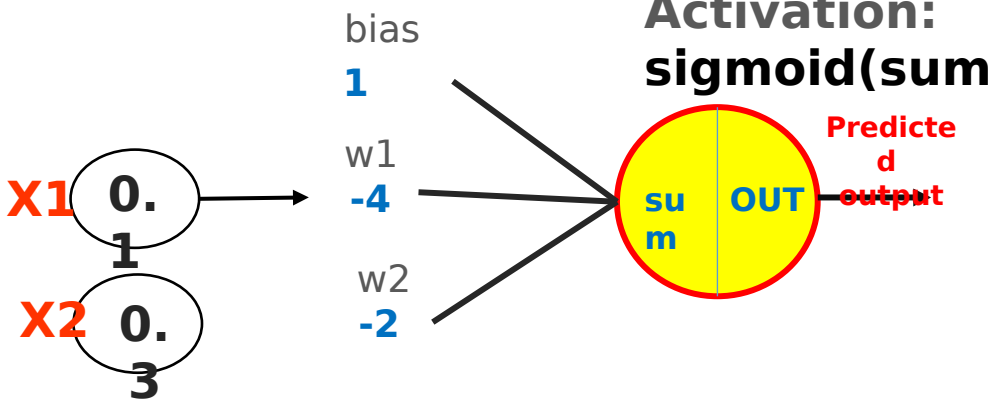
bias =bias+ (desired-predicted) * learning_rate

Fin Pour

Exemple:

X1	X2	output
0.1	0.3	0.03

Activation: sigmoid(sum)



sum=1*1 + (-4)*0.1+ (-2)*0.3 =0

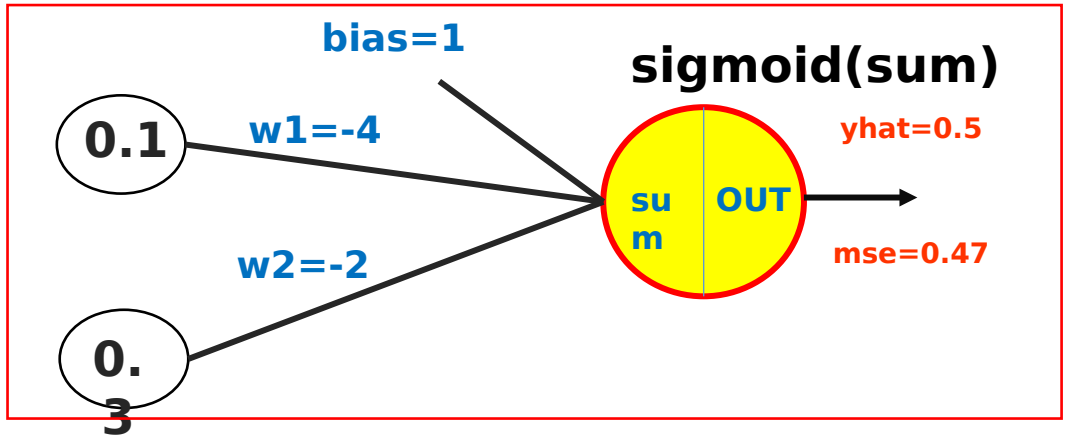
$$sigmoid(1.940) = \frac{1}{1+e^{-0}}$$ =0.5

$$error = (0.03 - 0.5) = -0.47$$

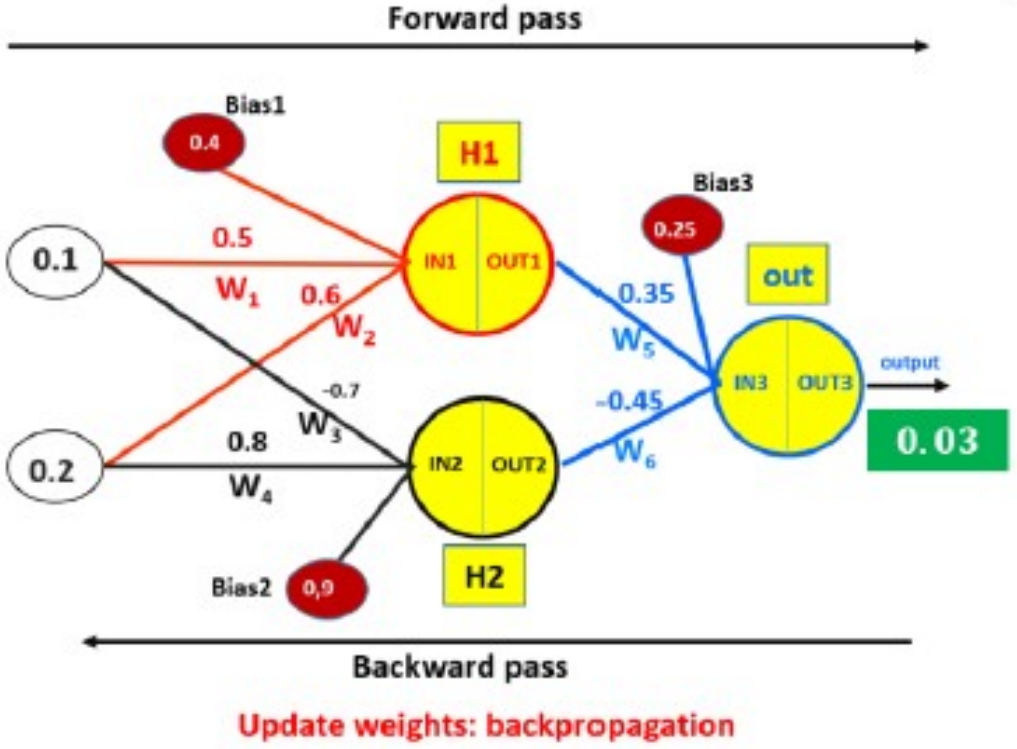
$$Cost = \frac{1}{n}(0.0.3 - 0.5)^2 = 0.2209$$

- Pour l'architecture dans l'exemple 2, le programme précédent ne peut pas connaitre les valeurs de mise à jour pour les poids des couches précédentes (couche cachée): bias1,w1,w2,w3,w4
  - L'erreur n'est pas exprimée en fonction de bias1,w1,w2,w3,w4
- Et en réalité, même dans l'exemple 1, l'erreur ne dépend pas directement de bias, w1 et w2
  - Mais plutôt elle dépend de **OUT=sigmoid(sum)**
  - **Car mse=(1/n)*(y-out)²**
- **Le recours à la règle de chaine rule est nécessaire**

Exemple 1



Exemple 2

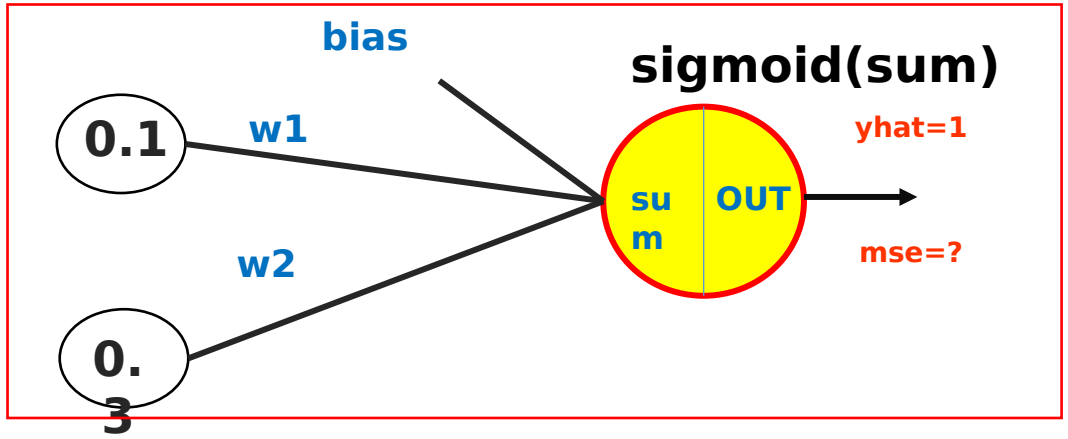


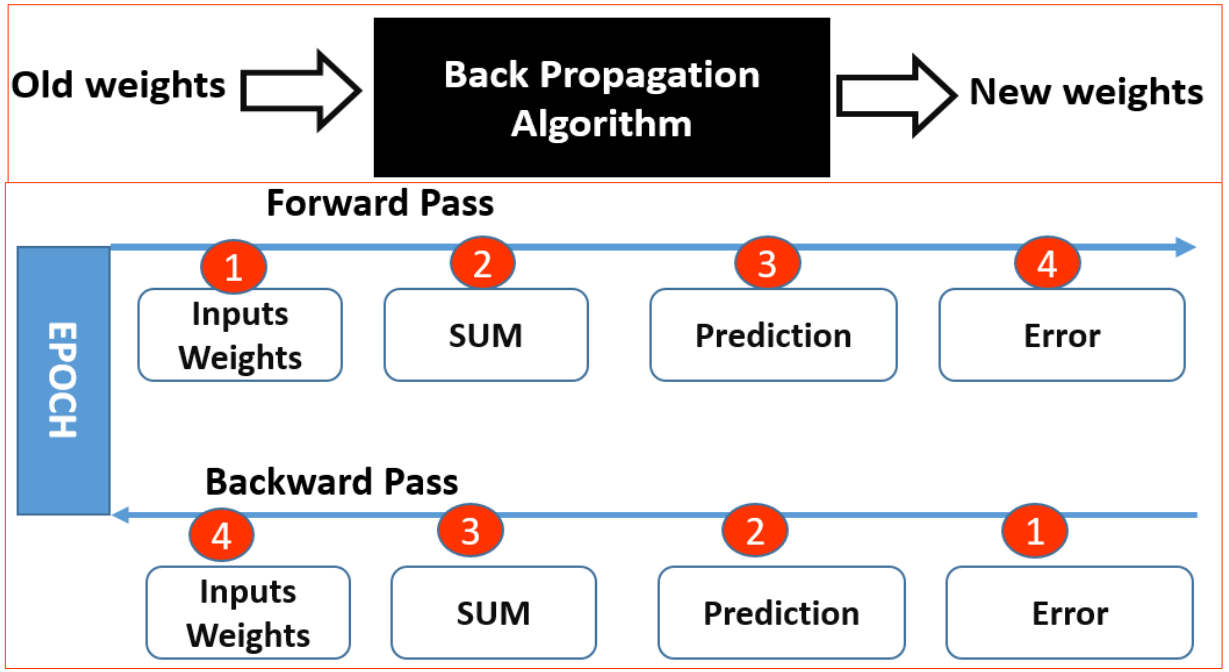
Backpropagation Algorithm

Backpropagation Algorithm

- Algorithme qui constitue un block important d'un Multilayer neural network pour l'apprentissage
- Il a été créé dans les années 1960,
- Devenu populaire par Rumelhart, Hinton and Williams dans un papier intitulé *"Learning re presentations by back-propagating errors"*

Exemple 1





1. Forward pass: pour la prédiction
2. Backward pass: pour la correction

Backpropagation Algorithm

- L'algorithme est utilisé pour entraîner efficacement un réseau de neurones à l'aide d'une méthode appelée **chain rule**
- Après chaque passage en avant (**Forward pass**) à travers un réseau, la rétropropagation effectue un passage en arrière (**Backward pass**) tout en ajustant les paramètres du modèle (weights and bias).

- **Comment un changement de poids va influencer sur l'erreur ?**

Dans la suite, pour simplifier nous allons utiliser:

$$\boldsymbol{Loss = \frac{1}{2}(desired - predicted)^2}$$

forward pass

Inputs

$x1, x2$

1 **Weights + bias**

$bias, w1, w2$

2 **SUM**

$$s = bias + w1 * x1 + w2 * x2$$

3 **Prediction**

$$predicted = f(s) = \frac{1}{1 + e^{-s}}$$

4 **Error**

$$Loss = \frac{1}{2}(desired - predicted)^2$$

On sait que les Wi et bias déterminent la valeur de Loss

- Quel est l'effet de W1 sur Loss? $\frac{\partial loss}{\partial w_1}$
- Quel est l'effet de W2 sur Loss? $\frac{\partial loss}{\partial w_2}$
- Quel est l'effet de bias sur Loss? $\frac{\partial loss}{bias}$

Connaitre l'effet pour la mise à jour des paramètres du modèle

$$\boxed{w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}}$$

$$\boxed{w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}}$$

$$\boxed{bias = bias - learning_rate * \frac{\partial loss}{\partial bias}}$$

Exprimer Loss en fonction de bias, w1 et w2

$$Loss = \frac{1}{2}(desired - predicted)^2$$

$$\text{Loss} = \frac{1}{2}\left(desired - \frac{1}{1+e^{-(w1*x1+w2*x2+bias)}}\right)^2$$

Pour connaitre L'effet de bias, w1 et w2 sur Loss

$$\frac{\partial Loss}{\partial w1} = \frac{\partial}{\partial w1}\left(\frac{1}{2}\left(desired - \frac{1}{1+e^{-(w1*x1+w2*x2+\text{bias})}}\right)^2\right)$$

$$\frac{\partial Loss}{\partial w2} = \frac{\partial}{\partial w2}\left(\frac{1}{2}\left(desired - \frac{1}{1+e^{-(w1*x1+w2*x2+\text{bias})}}\right)^2\right)$$

$$\frac{\partial Loss}{\partial bias} = \frac{\partial}{\partial bias}\left(\frac{1}{2}\left(desired - \frac{1}{1+e^{-(w1*x1+w2*x2+\text{bias})}}\right)^2\right)$$

Difficile à calculer

Autre alternative: chain rule

chain rule

Appelée aussi théorème de dérivation des fonctions composées
Ou règle de dérivation en chaîne

$y = f(u)$
et $u = g(x)$

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

https://en.wikipedia.org/wiki/Chain_rule

Inputs

$x1, x2$

4

Weights + bias

$bias, w1, w2$

3

SUM

$$s = w0 * 1 + w1 * x1 + w2 * x2$$

$$\frac{\partial s}{\partial w1}$$

$$\frac{\partial s}{\partial w2}$$

$$\frac{\partial s}{\partial bias}$$

2

Prediction

$$predicted = f(s) = \frac{1}{1 + e^{-s}}$$

$$\frac{\partial predicted}{\partial s}$$

1

Error

$$Loss = \frac{1}{2}(desired - predicted)^2$$

$$\frac{\partial Loss}{\partial predicted}$$

Loss n'est pas en fonction de w0,w1 et bias,
Mais on a:

Comment utiliser chain rule pour connaitre L'effet de bias, w1 et w2 sur Loss ?

$\frac{\partial Loss}{\partial w1}=?$ $\frac{\partial Loss}{\partial w2}=?$ $\frac{\partial Loss}{\partial bias}=?$

Utiliser chain Rule pour exprimer la variation de Loss en fonction de la variation de bias, w1 et W2

Inputs

$x1, x2$

4 **Weights + bias**

$bias, w1, w2$

3 **SUM**

$$s = w0 * 1 + w1 * x1 + w2 * x2$$

$$\frac{\partial s}{\partial w1}$$

$$\frac{\partial s}{\partial w2}$$

$$\frac{\partial s}{\partial bias}$$

2 **Prediction**

$$predicted = f(s) = \frac{1}{1 + e^{-s}}$$

$$\frac{\partial predicted}{\partial s}$$

1 **Error**

$$Loss = \frac{1}{2}(desired - predicted)^2$$

$$\frac{\partial Loss}{\partial predicted}$$

$$\frac{\partial Loss}{\partial w1} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w1}$$

$$\frac{\partial Loss}{\partial w2} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w2}$$

$$\frac{\partial Loss}{\partial bias} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial bias}$$

$$s = w1 * x1 + w2 * x2 + bias$$

$$\frac{\partial s}{\partial w1} = x1$$

$$\frac{\partial s}{\partial w2} = x2$$

$$\frac{\partial s}{\partial bias} = 1$$

$$predicted = f(s) = \frac{1}{1 + e^{-s}}$$

$$\frac{\partial predicted}{\partial s} = \frac{1}{1 + e^{-s}}\left(1 - \frac{1}{1 + e^{-s}}\right)$$

$$\frac{\partial predicted}{\partial s} = predicted\,(1 - predicted)$$

$$Loss = \frac{1}{2}(desired - predicted)^2$$

$$\frac{\partial Loss}{\partial predicted} = predicted - desired$$

```
def sigmoid (z):
    return 1/(1 + np.exp(-z))
def
sigmoid_derivative(predicted):
    return predicted * (1 -
predicted)
```

Algorithme

De 1 à n_epochs faire

- Forward pass
- Backward pass

$$\frac{\partial Loss}{\partial w1} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w1}$$

W_i =w_i+ (desired-predicted)* Xi * learning_rate

Initialize learning rate

1. **Initialize weights**

De 1 à n_epochs faire

2. $$sum = \sum_{i=0}^{n} W_i * X_i$$

3. **Activation function →**

$$f(s) = \frac{1}{1+e^{-s}} = output$$

4. **error= desired-pre**

$$Loss = \frac{1}{2}(desired - predicted)^2$$

5. **Weights update**

$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

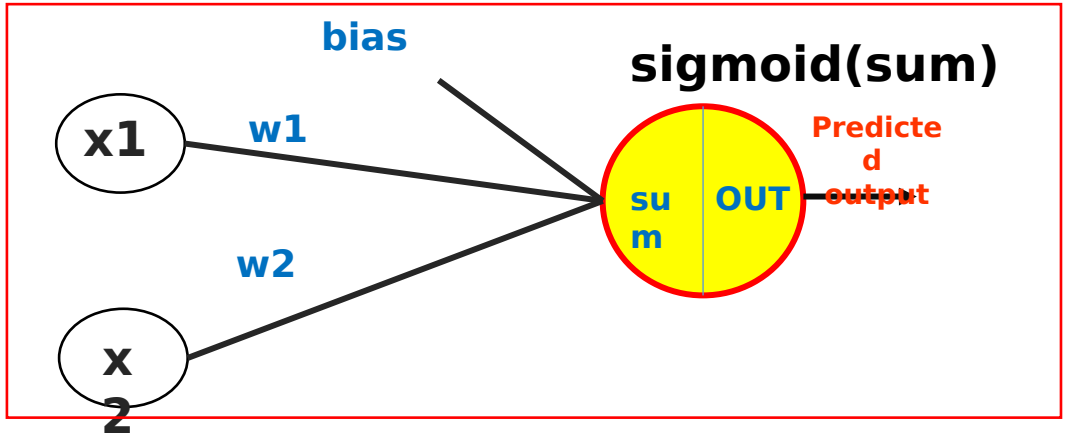
$$bias = bias - learning_rate * \frac{\partial loss}{\partial bias}$$

Back propagation algorithm

Exemple d’application numérique:

X1	X2	output
0.1	0.3	0.03
…		

learning_rate=0.01



- Initialiser les poids de manière aléatoire
- Mettre à jour les poids en appliquant l’algorithme de backpropabation pour minimiser la fonction Loss
- Tourner l’algorithme en deux itérations

X1	X2	output
0.1	0.3	0.03
...		

learning_rate= 0.01

Forward pass

Sum=w1*x1+w2*x2+bias

Predicted output=sigmoid(sum)

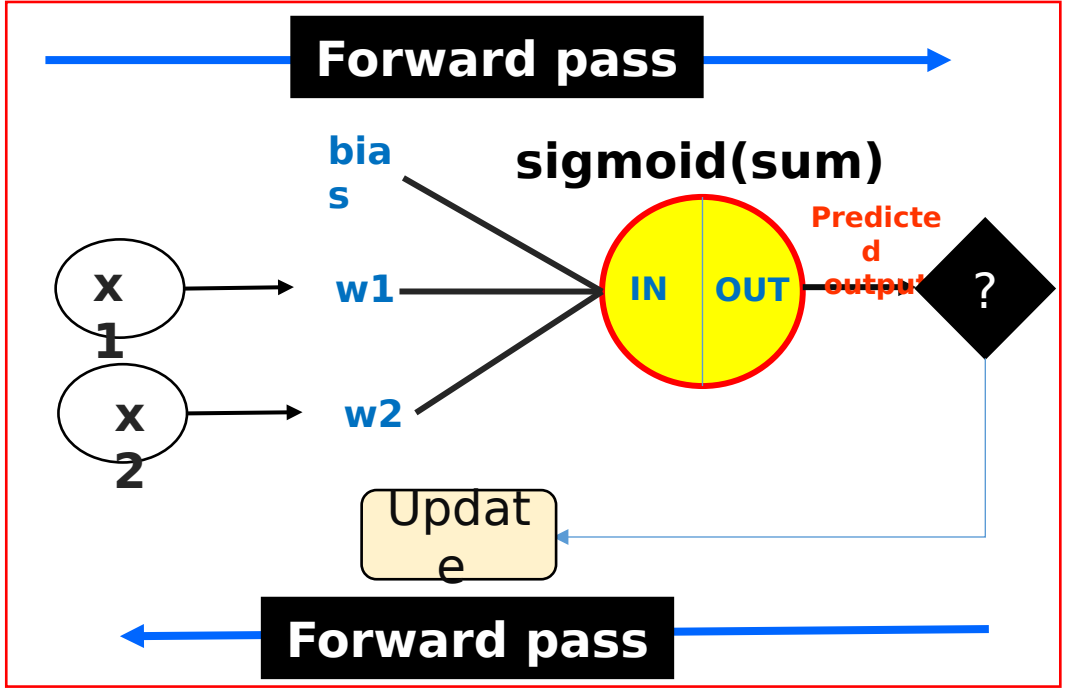
$$Loss = \frac{1}{2}(desired - predicted)^2$$

Backward pass

$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

$$bias = bias - learning_rate * \frac{\partial loss}{\partial bias}$$



$$\frac{\partial Loss}{\partial w1} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w1}$$

$$\frac{\partial Loss}{\partial w2} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w2}$$

$$\frac{\partial Loss}{\partial bias} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial bias}$$

$$\frac{\partial Loss}{\partial predicted} = predicted - desired$$

$$\frac{\partial predicted}{\partial s} = f(s)\,(1 - f(s))$$

$$\frac{\partial s}{\partial w1} = x1$$

$$\frac{\partial s}{\partial w2} = x2$$

$$\frac{\partial s}{\partial bias} = 1$$

Itération 1

	X1	X2	output
Entree 1	0.1	0.3	0.03
Entree 2	...		

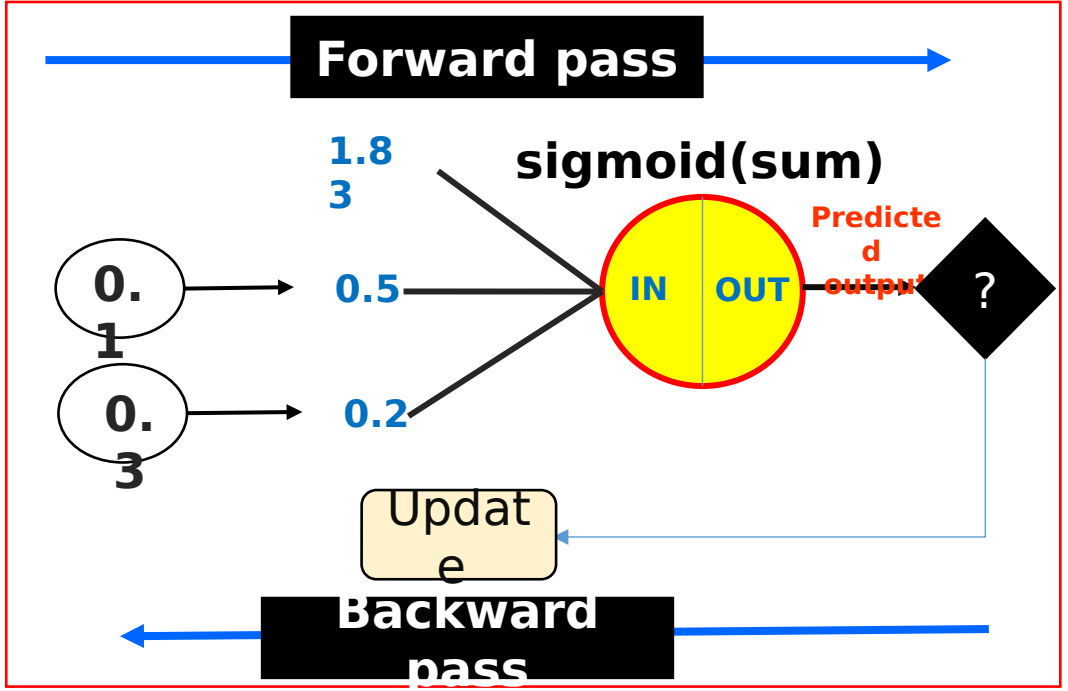
learning_rate= 0. 1

Forward pass 1

Sum=w1*x1+w2*x2+bias =1.940

Predicted output=sigmoid(sum) =0.874

$$Loss = \frac{1}{2}(desired - predicted)^2 = 0.3564$$



Backward pass 1

$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

$$w_1 = 0.5 - 0.01 * 0.0092 = 0.499$$

$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

$$w_2 = 0.2 - 0.01 * 0.0278 = 0.197$$

$$bias = bias - learning_rate * \frac{\partial loss}{\partial bias}$$

$$bias = 1.83 - 0.01 * 0.0927 = 1.821$$

$$\frac{\partial Loss}{\partial w1} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w1}$$ 0,0092

$$\frac{\partial Loss}{\partial w2} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w2}$$ 0,0278

$$\frac{\partial Loss}{\partial bias} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial bias}$$ 0,0927

$$\frac{\partial Loss}{\partial predicted} = predicted - desired$$ 0.844

$$\frac{\partial predicted}{\partial s} = f(s)\,(1 - f(s))$$ 0.11

$$\frac{\partial s}{\partial w1} = x1$$ 0.1

$$\frac{\partial s}{\partial w2} = x2$$ 0.3

$$\frac{\partial s}{\partial bias} = 1$$ 1

Itération 2

	X1	X2	output
Entree 1	0.1	0.3	0.03
Entree 2	...		

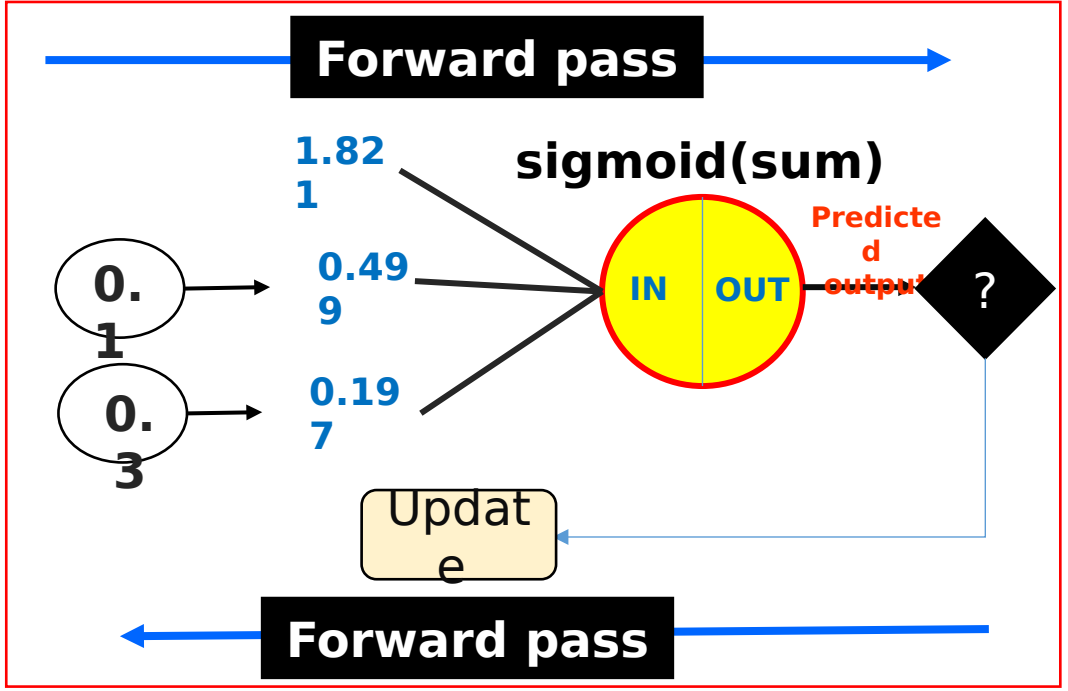
learning_rate= 0.01

Forward pass 2

Sum=w1*x1+w2*x2+bias =1.929

Predicted output=sigmoid(sum) =0.873

$$Loss = \frac{1}{2}(desired - predicted)^2 = 0.3555$$



Backward pass 2

$$w_1 = w_1 - learning_rate * \frac{\partial loss}{\partial w_1}$$

$w_1 = ?$

$$w_2 = w_2 - learning_rate * \frac{\partial loss}{\partial w_2}$$

$w_2 = ?$

$$bias = bias - learning_rate * \frac{\partial loss}{\partial bias}$$

$bias = ?$

$$\frac{\partial Loss}{\partial w1} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w1}$$?

$$\frac{\partial Loss}{\partial w2} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial w2}$$?

$$\frac{\partial Loss}{\partial bias} = \frac{\partial Loss}{\partial predicted} * \frac{\partial predicted}{\partial s} * \frac{\partial s}{\partial bias}$$?

$$\frac{\partial Loss}{\partial predicted} = predicted - desired$$?

$$\frac{\partial predicted}{\partial s} = f(s)\,(1 - f(s))$$?

$$\frac{\partial s}{\partial w1} = x1$$ 0.1

$$\frac{\partial s}{\partial w2} = x2$$ 0.3

$$\frac{\partial s}{\partial bias} = 1$$ 1

Backpropagation – Algorithm (version1: utiliser des variables simples)

```
Epoch 10000: ------------------------------------
sum:-3.2627453450687973
prediction: 0.036871592334909074
Loss-quadr: 2.360939060859058E-5
0,027   -1,219 -2,900
-----------------------------------------------
```

backpropagation

Backpropagation – Algorithm (version1: utiliser des matrices)

```
Epoch 10000: ------------------------------------
sum:-3.2627453450687973
prediction: 0.036871592334909074
Loss-quadr: 2.360939060859058E-5
0,027   -1,219 -2,900
-----------------------------------------------
```

backpropagation